Two flows, step by step

How a business user's question is answered from the library, and how a new document or version gets into it.

For someone new to AI and RAG, with every tool named.

Knowledge Portal | 16 October 2026
Companion to "RAG explained from zero". Facts come from the repository's code.

Contents

1. Who and what takes part **4**

2. Flow A: a business user asks, the library answers **6**

Step by step . . . 7

Inside the search engine . . . 10

When something goes wrong . . . 11

3. Flow B: a document enters the library **12**

Step by step . . . 13

A new version of an existing document . . . 17

When something goes wrong . . . 18

Withdrawing a document . . . 18

4. What keeps both flows safe and correct **19**

5. Glossary **20**

How to read this guide

This guide follows two journeys through the knowledge portal, from the first click to the final result, one step at a time:

- **Flow A, answering.** A business user writes a requirement. The requirement portal asks the knowledge portal for evidence, and the knowledge portal searches its reference library and sends back the best passages with exact citations.
- **Flow B, learning.** A knowledge admin uploads a new document, or a new version of an existing one. It is scanned, read, checked by a person, cut into chunks, turned into numbers and switched into service, so Flow A can find it.

It is written for someone new to AI. Every step says what happens in plain words, which tool does it, and why it is done that way. If a word like *embedding*, *chunk* or *vector* is new to you, the companion guide *RAG explained from zero* (docs/guides/rag-explained-from-zero.pdf) introduces it from the beginning; the glossary at the end of this guide repeats the short definitions.

Think of it like this

Think of the portal as a reference library with two doors. Through the **front door** (Flow A) people ask questions and a librarian brings back the right pages, with page numbers. Through the **back door** (Flow B) new books arrive: they are checked for damage, read, approved by an editor, catalogued, and only then put on the shelf.

Good to know

Everything about the knowledge portal comes from this repository's code. The requirement portal is a separate service with its own repository: this guide describes what it sends and receives, but not how it writes its own AI prompts, which is not in this repository.

1. Who and what takes part

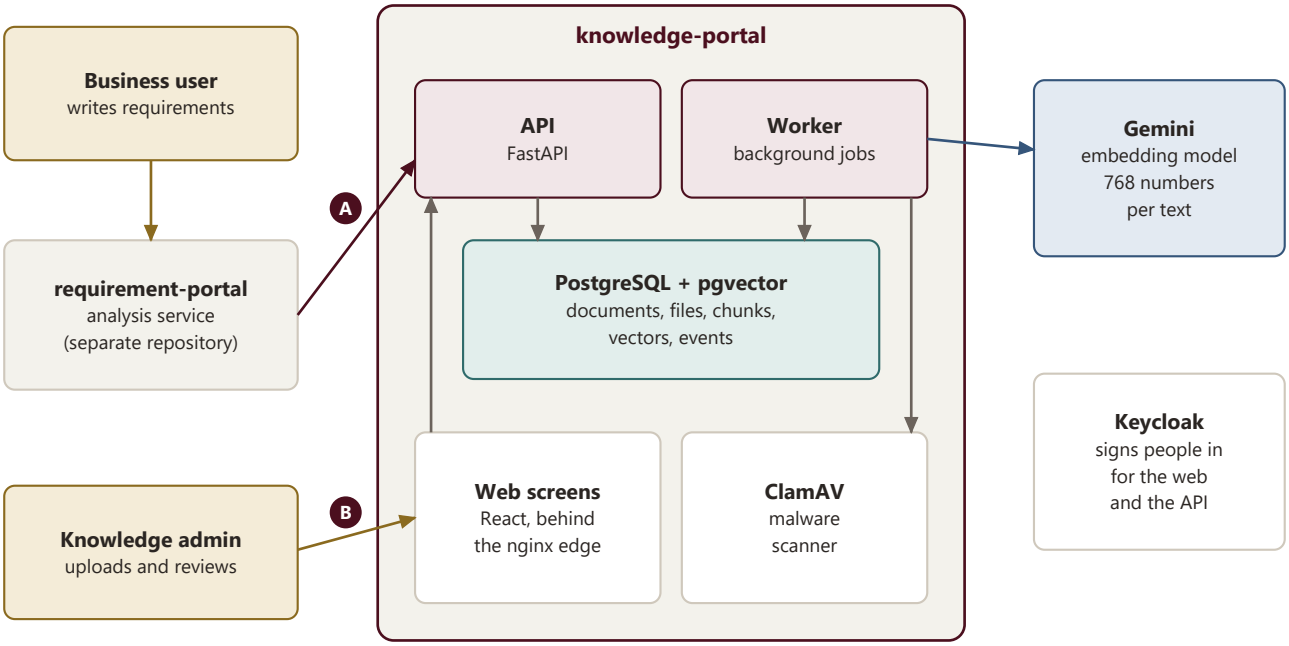


Figure 1. The parts of the platform. A is Flow A, a question arriving from the requirement portal; B is Flow B, a document arriving from a knowledge admin.

Two kinds of people and three services take part. The table lists every tool you will meet in the steps, and the one job each does here.

Tool	What it is	Its job in these flows
requirement-portal	Our sister service, a separate application	Where business users write requirements. It calls the knowledge portal for evidence and uses AI to analyse the requirement with that evidence.
Knowledge portal API	A web service written in Python with **FastAPI**	Receives every request, checks who is calling, and runs the right use case.
Worker	A second Python process running in the background	Does the slow work of Flow B, so uploads return at once: scanning, reading and indexing documents.
PostgreSQL 17	A relational database	Stores documents, their files, reviews, approvals, chunks and events. One database for everything.
pgvector + HNSW	A PostgreSQL extension and its index	Stores each chunk's 768 numbers and finds the nearest ones to a question quickly.
PostgreSQL text search	Built-in keyword search (tsvector)	Finds chunks that share words with the question.
Gemini embedding model	Google's gemini-embedding-001, reached over the internet	Turns a piece of text into 768 numbers that describe its meaning.
ClamAV	An open-source malware scanner, in its own container	Checks every uploaded file before anything else reads it.
Document extractors	Readers from our platform-kernel library (pypdf, pypdfium2, openpyxl and Office XML readers)	Pull the text out of PDF, Word, PowerPoint, Excel, CSV, text and Markdown files, block by block.

Tool	What it is	Its job in these flows
Keycloak	A sign-in service (OIDC)	Signs people in; their token carries their role, such as knowledge_admin.
Service token	A secret credential for one service calling another	Proves a call to /internal/... really comes from the requirement portal.
React web screens and nginx	The browser application and the web server in front of it	Where knowledge admins upload, review and approve.
Docker Compose	Runs each part in its own container	Starts the API, worker, database, scanner and web server together in production.

2. Flow A: a business user asks, the library answers

The short version. The requirement portal sends the requirement's text to the knowledge portal. The knowledge portal turns the text into numbers, runs two searches over its approved chunks (one by words, one by meaning), merges them, checks every candidate is still approved, keeps the best few and returns them with exact citations. The requirement portal then uses those passages as evidence for its AI analysis and shows the answer with its sources.

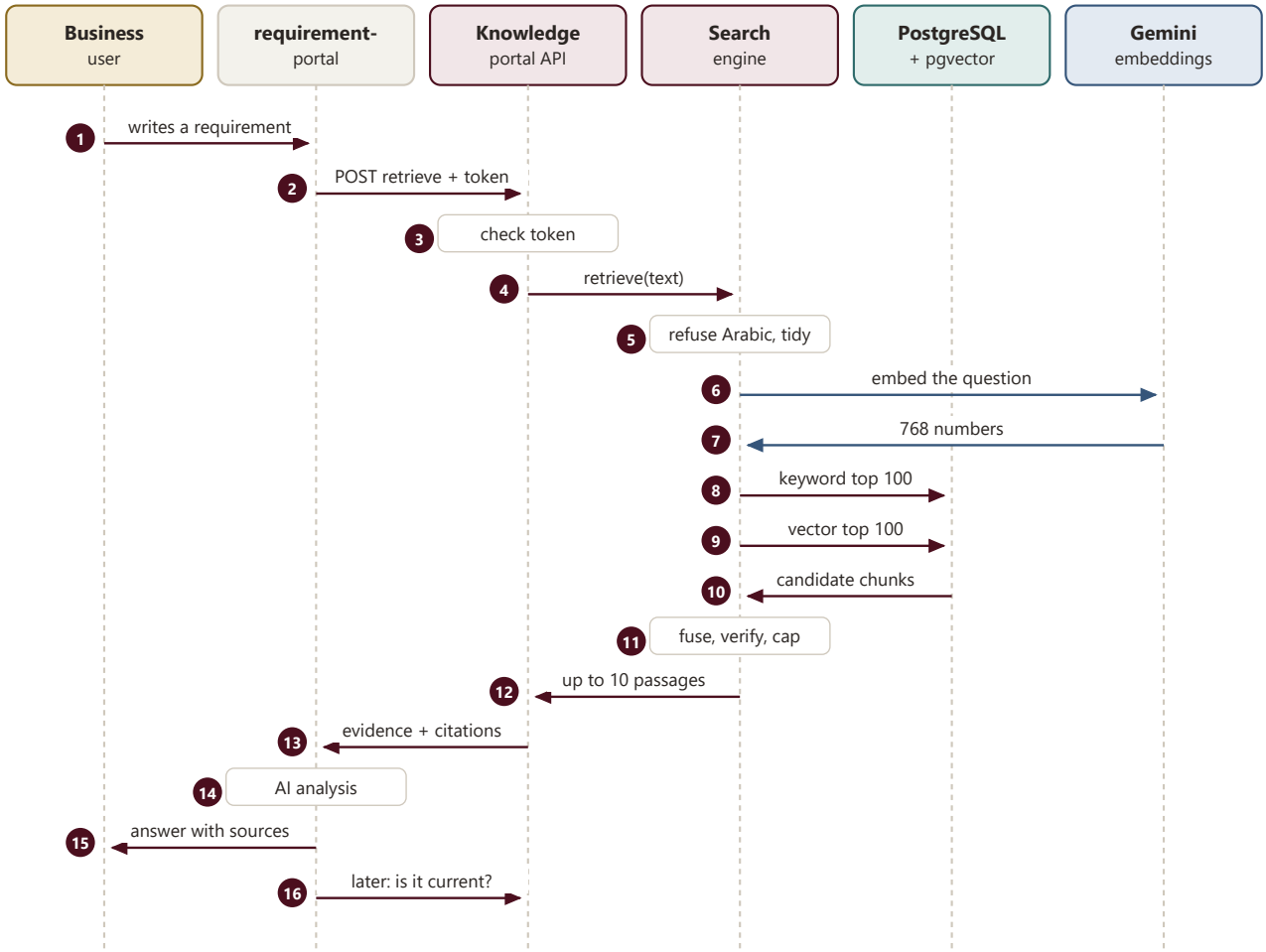


Figure 2. Flow A as a sequence. Time runs down the page; each numbered arrow is one step below. Blue arrows go to the model provider.

> **Key idea**
>
> The knowledge portal never writes the answer. It finds and returns evidence. The writing happens in the requirement portal, with the passages in front of its AI model, so every claim can point at a passage someone can open.

Step by step

Step 1: A business user writes a requirement

In the requirement portal, an analyst or product owner writes or opens a requirement, for example: "*Must sales check fibre coverage before giving a quote?*" When the requirement is analysed, the requirement portal needs to know what the business's own documents say about it.

Tool: requirement-portal (separate service).

Step 2: The requirement portal calls the knowledge portal

It sends an HTTP request: `POST /internal/library/retrieve` with a small JSON body, `{"query": "...the requirement text..."}`. The text can be up to 2,000 characters. The request carries a service token. There is also `/internal/library/search`, which returns the full ranked list rather than a prompt-sized selection.

Why: The /internal routes are a contract between the two services. They change only by adding, never by breaking what the other side relies on.

Tool: HTTP and JSON over the internal network; the service token.

Step 3: The API checks who is calling

Before any search, a guard on every /internal route checks the service token. A request without a valid token is refused with 401 and nothing else runs. A second check in the route itself refuses again if the first was somehow skipped.

Why: Library content is internal. Only the requirement portal may read it this way, and the token proves the caller is the requirement portal.

Tool: FastAPI middleware and platform-kernel's internal route guard.

Step 4: The API hands the text to the search engine

The route calls the library's retrieval use case with the text. If no document is in service yet, it returns an empty list at once.

Tool: ReferenceKnowledge.retrieve, our application code.

Step 5: The text is checked and tidied

Arabic text is refused with a clear message (status 422): the library holds English documents only. The text is then normalised: unusual character forms are unified, letters are made lower case and extra spaces are removed. Chunks were normalised the same way when they were stored, so both sides look alike.

Why: Comparing "Coverage" with "coverage " should not fail on capitals or spaces.

Tool: Our normalisation function (normalize_search) and the English-only rule.

Step 6: The question becomes 768 numbers

The tidied text is sent to the embedding model, which returns a list of 768 numbers: its **embedding**. Texts with similar meaning get similar numbers. The system checks the answer is usable: exactly 768 numbers, all finite, not all zero.

Why: This is how search can match "encrypted" with "scrambled": by meaning, not spelling. The chunks were embedded by the same model, so both are on the same map of meaning.

Tool: Google Gemini, model gemini-embedding-001, through our embedding adapter (chosen in config/llm.yaml).

Step 7: The numbers come back

The embedding returns to the search engine. This is the only call to an AI model in the whole of Flow A inside the knowledge portal.

Tool: Gemini embedding model.

Step 8: Keyword search

The database looks for chunks containing the question's words: up to 24 distinct words, any of them may match. It ranks the matches with PostgreSQL's ranking function (ts_rank_cd) and keeps the top 100.

Why: Keyword search is precise on exact names and codes, such as "XGPON" or "SIP Trunk 30", where meaning alone can blur.

Tool: PostgreSQL text search over a stored tsvector column.

Step 9: Meaning search

The database also finds the 100 chunks whose 768 numbers are closest to the question's, measured by cosine distance (the angle between the two lists of numbers).

Why: Meaning search finds passages that say the same thing in different words.

Tool: pgvector, with an HNSW index so it does not compare against every chunk one by one.

Step 10: Candidates come back

Up to 200 candidate chunks return, each with its text, its label (document title, section, location) and its place in each ranking. Only chunks of each document's version in service are eligible: drafts, older versions and withdrawn documents are never searched.

Tool: PostgreSQL.

Step 11: Fuse, verify and diversify

Four things happen in the search engine. **Fuse:** the two rankings are merged by reciprocal rank fusion; a chunk earns 1/(60 + its rank) from each list, and equal scores are ordered by the chunk's content hash, so the order is the same every time. **Verify:** each candidate is rebuilt from the approved text and must match exactly; its document must still be in service with the same approval. **Diversify:** duplicates are dropped, and each document may contribute at most 3 chunks, from at most 10 documents. **Budget:** for the requirement portal's prompt, at most 10 passages and about 2,100 tokens are kept.

Why: Verification means the portal never hands out text that differs from what a person approved. Diversity stops one long document from crowding out the others. The budget keeps the AI prompt small enough to be fast and focused.

Tool: Our search code (ReferenceKnowledge.search and retrieve).

Step 12: Passages go back to the API

Each kept chunk becomes one piece of **evidence**: an exact citation plus a little surrounding text from the same section (its neighbours), so the AI sees the sentence before and after.

Tool: ReferenceEvidence objects in our application code.

Step 13: The API replies to the requirement portal

The answer is a JSON list of evidence items. Each citation names the document, its title, version, approval, block, location (for example "Table 1, row 3" or "Page 2"), the exact quoted text, its start and end inside the approved passage, and a fingerprint of the text (a lineage hash).

Tool: FastAPI, JSON.

Step 14: The requirement portal writes its analysis

In the requirement portal, the passages are given to its AI model as evidence for analysing the requirement. How that prompt is built lives in the requirement portal's own code, not in this repository.

Why: Grounding the analysis in approved company text is the whole point of RAG: answers come from your documents, not from the model's memory.

Tool: requirement-portal and its AI model (outside this repository).

Step 15: The business user sees the answer with its sources

The user reads the analysis and can open each cited passage: which document, which page or row, and the exact words.

Tool: requirement-portal screens.

Step 16: Later: is the cited passage still current?

Documents change. Before relying on a citation again, the requirement portal can ask `GET /internal/library/passages` with the citation's identifiers. The passage is returned only while its approval is still in service; otherwise the answer is 409, meaning *replaced or withdrawn*. It also keeps its own copy of each document's state up to date by reading the events the knowledge portal writes (Flow B, step 18).

Why: A requirement approved last month may cite a policy that changed this week. This check is how stale evidence is caught.

Tool: The /internal/library/passages route and the /internal/events outbox.

Inside the search engine

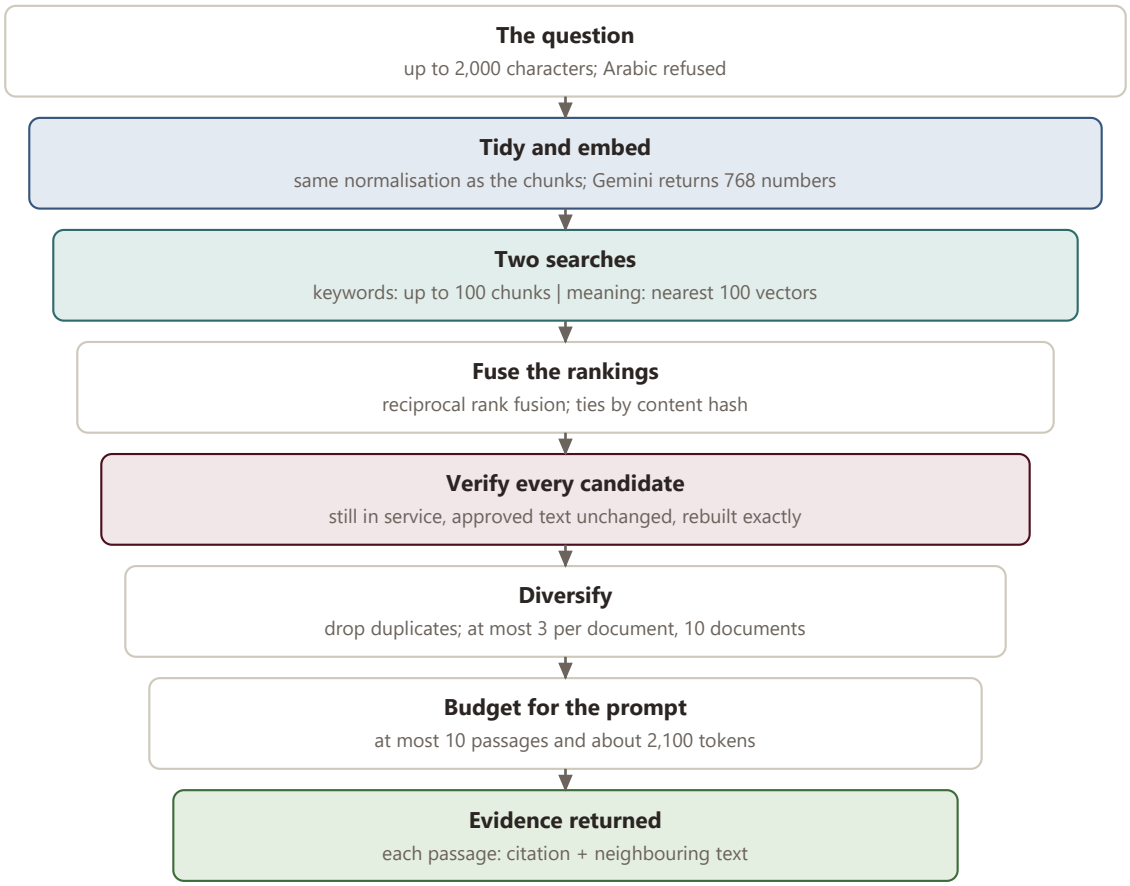


Figure 3. From one question to a handful of passages. Each layer removes or reorders candidates; only verified, approved text survives.

A small worked example with the offline test models: the question "*Must sales check fibre coverage before giving a quote?*" brings back, first, "Confirm XGPON coverage at the customer's address before quoting." from the *Bundle ordering procedure*, Line 2, and second, "High-speed bundles require XGPON coverage at the customer address." from the *XGPON coverage rules*, Line 1. Both are cited with their document, version and line, so the analyst can open them.

Part of one evidence item	What it holds	Why it is there
Citation: document, version, approval	Which document, which uploaded version, which approval	To show where the text came from, and to check later that it is still current.
Citation: block and location	"Line 2", "Page 2", "Table 1, row 3", "Slide 2, paragraph 3"	So a person can find the exact spot in the original.
Citation: excerpt and offsets	The exact approved words and their position	Quotes are exact, never paraphrased by the search.
Citation: lineage hash	A fingerprint of the excerpt	Detects any change to the cited words.
Context text	The chunk with its neighbours from the same section	Gives the AI enough surrounding text to understand the passage.

When something goes wrong

Situation	What the caller gets	Why
No service token, or a wrong one	401, nothing searched	Only the requirement portal may call /internal routes.
Arabic text, or more than 2,000 characters	422 with a clear message	The library is English-only; very long text is not a question.
No document in service yet	An empty list	There is nothing to search.
The embedding provider fails or answers badly	An error; nothing half-done is returned	A wrong vector must never decide what is evidence.
The search model changed but documents were not rebuilt	An error naming the index generation	Vectors from two different models are never compared.
A cited passage was replaced or withdrawn	409 from /internal/library/passages	Stale evidence is caught before it is relied on.

3. Flow B: a document enters the library

The short version. A knowledge admin uploads a file. The API checks it and saves it, then answers at once. In the background, the worker scans the file for malware, reads its text into blocks and marks it ready for review. The owner checks every block and approves. The worker then cuts the approved text into chunks, gets their 768 numbers from the embedding model, stores them out of sight, checks everything arrived, and switches the document into service in one save. From that moment, Flow A can find it.

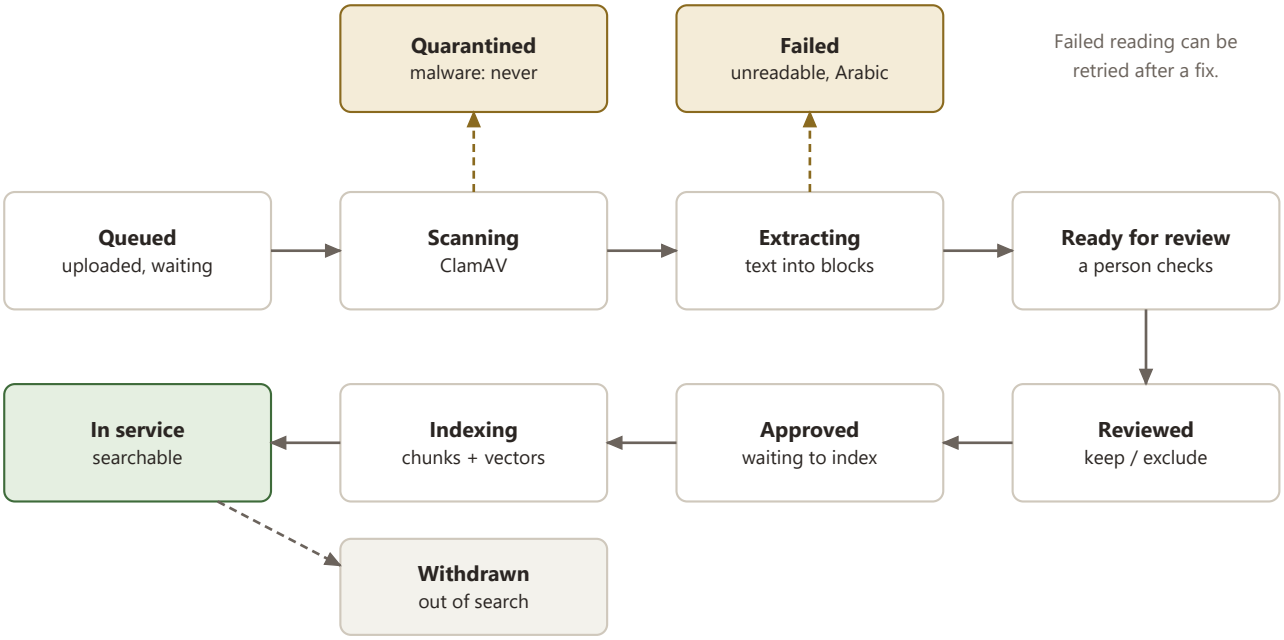


Figure 4. The life of one uploaded version. The main path runs along the top row and back along the second; amber boxes are the exits for files that cannot be used.

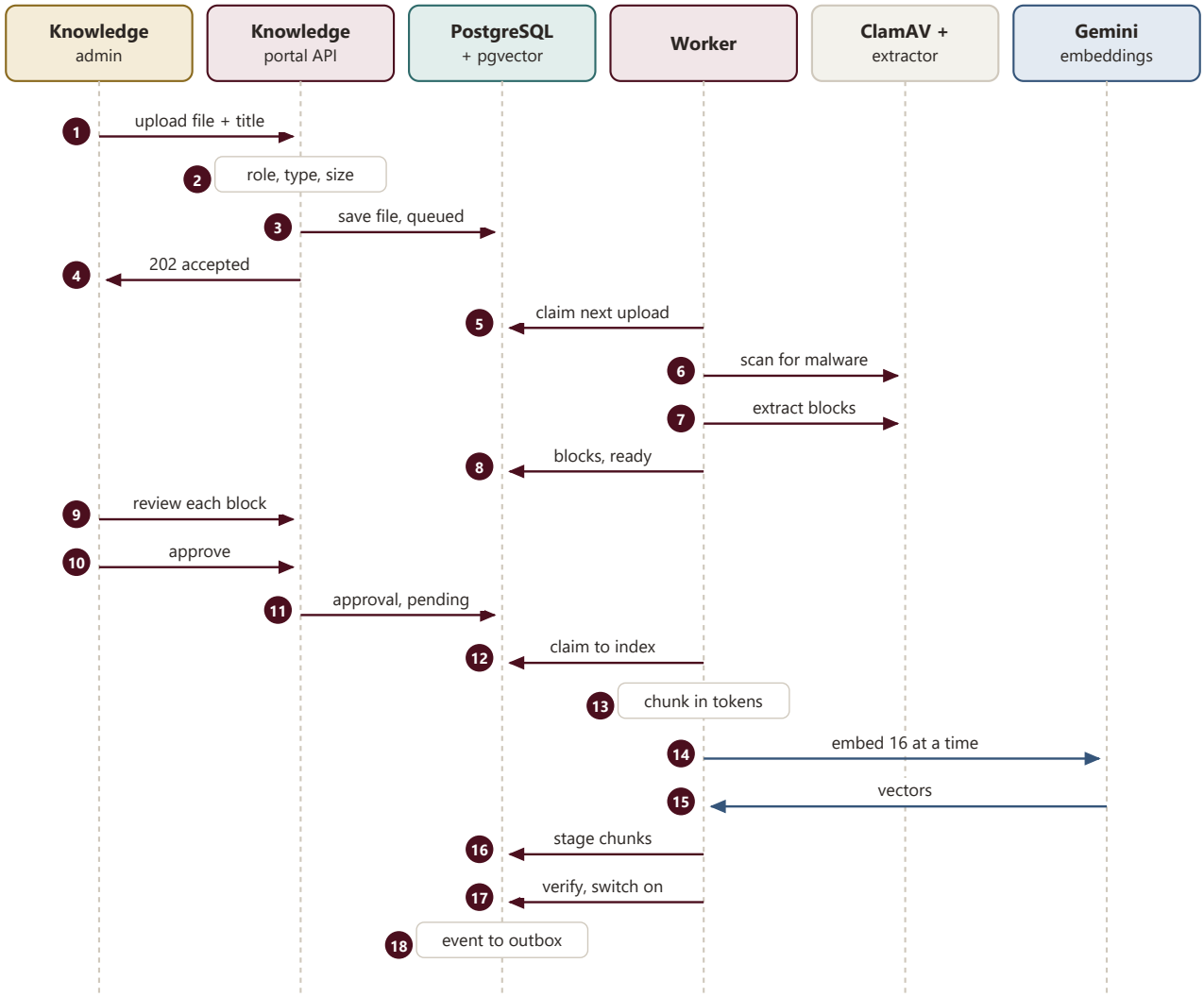


Figure 5. Flow B as a sequence. The API answers the admin at step 4; the slow work happens in the worker, steps 5 to 18.

Key idea

Nothing machine-read reaches search without a person approving it. Extraction is automatic, but the owner decides block by block what may be used, and the approval is tied to exactly what they reviewed.

Step by step

Step 1: A knowledge admin uploads a file

In the web screens, the admin chooses a file and gives it a title. The browser sends `POST /library/ingestions` with the file, the title and an **idempotency key**, a unique id for this upload attempt. To add a new version of an existing document, the same request also names the document and the version number the admin was looking at.

Tool: React web screens, nginx, the FastAPI API, and the admin's Keycloak sign-in token.

Step 2: The API checks the upload

The caller must be signed in with the knowledge_admin role. The file must be a supported type (PDF, Word, PowerPoint, Excel, CSV or TSV, text, Markdown, PNG or JPEG) whose content matches its declared type, and at most the configured size (10 MB by default). The title must be present, at most 200 characters, and contain no Arabic. Sending the same idempotency key again returns the same document instead of a duplicate; the same key with different content is refused.

Why: A double click or a network retry must never create two documents.

Tool: Our upload validation and the DocumentLibrary.submit use case.

Step 3: The file and a new version are saved

The file's bytes are stored, with a SHA-256 checksum (a fingerprint of the file). A new document is created, or a new version is added to an existing one, in the **queued** state. Adding a version only succeeds if nobody changed the document since the admin loaded it; otherwise they are asked to reload.

Why: The version number check (optimistic concurrency) stops two people silently overwriting each other.

Tool: PostgreSQL, in one transaction.

Step 4: The admin gets an answer at once

The API replies 202 Accepted with the document's current view. The admin can leave the page; the rest happens in the background and the screen shows each state as it changes.

Why: Reading a large file can take minutes. Nobody should wait on an open request for that.

Tool: FastAPI.

Step 5: The worker claims the upload

The worker checks for work about once a second when idle. It claims the oldest queued version with a **lease**: a reservation that lasts 11 minutes. If the worker crashes, the lease runs out and the version is claimed again.

Why: Work is never lost and never done twice at the same time.

Tool: The worker process (knowledge_portal.interfaces.worker), PostgreSQL.

Step 6: The file is scanned for malware

Before any reader opens the file, ClamAV scans its bytes. A file that fails is **quarantined**: it can never be read, retried or published.

Why: Readers for complex formats are a common attack route. Nothing unscanned reaches them.

Tool: ClamAV, in its own container (an offline stand-in is used only in development).

Step 7: The text is extracted into blocks

A reader for the file's format pulls out its text as **blocks**: paragraphs, headings, table rows, slide paragraphs, spreadsheet rows, PDF pages. Each block keeps its location, such as "Paragraph 4" or "Table 1, row 3". The reader runs in a separate process with hard limits: 10 minutes, 4 GB of memory, and caps on pages, characters, cells and image size. Problems are recorded as warnings; some are **blocking**, for example a chart whose content cannot be read as text. If any block contains Arabic, reading fails and names the first Arabic location.

Why: A broken or hostile file must not take down the service. The limits contain it.

Tool: platform-kernel's extractors in a bounded subprocess: pypdf and pypdfium2 for PDF, openpyxl for Excel, Office XML readers for Word and PowerPoint, text and CSV readers.

Step 8: Ready for review

The blocks and warnings are saved and the version becomes **ready for review**. If the file could not be read, it is **failed**, with the reason, and can be retried once fixed.

Tool: PostgreSQL.

Step 9: The owner reviews every block

The document's owner, or an admin with a live grant to act for them, sees each block next to a preview of the original. For every block they decide: **include** it, possibly correcting the extracted text, or **exclude** it with a reason (private, wrong, unreadable). Every block must be decided, including images. The review is saved as a **revision** with an explanation, and gets a fingerprint.

Why: Machines misread tables, footers and scans. A person decides what the library will say.

Tool: Web review screen; POST /library/documents/{id}/versions/{version}/review.

Step 10: The owner approves

Approval names the exact revision and its fingerprint. It is refused if the revision is not the latest, if the fingerprint does not match what was reviewed, if any blocking warning remains on an included block, if no block is included, or if someone else changed the document meanwhile.

Why: What is approved is exactly what was reviewed, and only the owner (or someone acting for them, recorded as such) can approve.

Tool: POST /library/documents/{id}/versions/{version}/approval, the domain approval rules.

Step 11: An approval is recorded, waiting to be indexed

A **publication** is saved: this version, this revision, this fingerprint, approved by this person at this time. It is not searchable yet.

Tool: PostgreSQL.

Step 12: The worker claims the approval

The worker's second job picks up approvals waiting to be indexed, again under an 11-minute lease, and counts the attempt.

Tool: The worker, PostgreSQL.

Step 13: The approved text is cut into chunks

Each included block is cut into chunks of about 135 and at most 200 approximate tokens, breaking at sentence ends where possible and never mixing two blocks. Each chunk gets a label (document title, section, location), and a table row is searched with its table's column names, such as "Pooled data: 25 GB per line". The cited text itself is never changed.

Why: Small, labelled chunks match questions sharply and can be cited exactly.

Tool: Our structure-aware chunker (StructureAwareChunks) and approximate token counter.

Step 14: The chunks are embedded, 16 at a time

Chunks are sent to the embedding model in batches of 16. Before sending, the worker looks each chunk up by its content hash: if exactly the same text was embedded before (for example, an unchanged paragraph of a previous version), its stored numbers are reused and not paid for again.

Why: Re-indexing a new version where little changed costs little.

Tool: Gemini embedding model (gemini-embedding-001), our embedding cache.

Step 15: The vectors come back and are checked

Each vector must have exactly 768 finite numbers that are not all zero. The model must return exactly one vector per chunk. Anything else fails the attempt.

Tool: Our vector validation.

Step 16: Chunks are stored out of sight

Each batch of chunks and vectors is written to the chunk table. They are **staged**: search ignores them, because only chunks of the version in service are eligible.

Why: If indexing stops halfway, nothing half-built is ever searched, and finished batches are kept for the retry.

Tool: PostgreSQL with pgvector; the keyword index is computed from each chunk's text.

Step 17: Everything is verified, then switched on

The worker checks that the stored chunks match exactly what the chunker produced (the manifest: every chunk id with its content hash). Then, in one save, the version becomes the one **in service**: the document's "published" pointer moves to it. From this moment Flow A finds the new text, and the previous version, if any, no longer appears.

Why: One atomic switch means searchers see either the old version or the new one, never a mix.

Tool: PostgreSQL transaction.

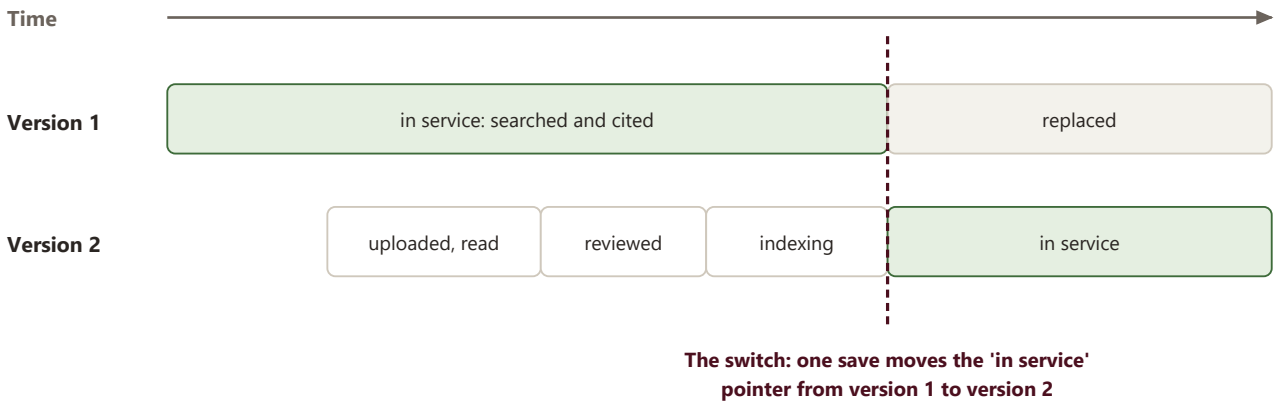
Step 18: An event tells the requirement portal

Every change to a library document also writes an event, `reference_document_changed`, into an outbox in the same transaction. The requirement portal reads new events from `/internal/events` and updates its own copy of each document's state, including when it is due for review again (every 180 days by default).

Why: The event is saved together with the change, so the requirement portal can never miss a change or hear about one that did not happen.

Tool: The outbox table in PostgreSQL and the /internal/events route.

A new version of an existing document



Search never sees a half-built version: new chunks stay invisible until the switch.

Figure 6. Replacing version 1 with version 2. Version 1 keeps answering questions until the very moment version 2 is switched on.

A new version follows exactly the same steps as a new document, with four differences:

- **Upload:** the request names the existing document and the version number the admin saw. If someone else changed it meanwhile, the upload is refused and the admin reloads.
- **No gap in service:** the old version stays in service through scanning, reading, review and indexing. Only the final switch (step 17) replaces it.
- **Cheaper indexing:** chunks whose text and label did not change reuse their stored numbers (step 14).
- **Citations move on:** passages cited from the old version are no longer current. The requirement portal learns this from the event (step 18), and /internal/library/passages answers 409 for them, so it can re-check the affected requirements.

> **Good to know**
> There is a second, more careful way to put a version in service, used for documents with large tables: the owner previews how it will be chunked, starts a **build**, and once the worker has indexed it, activates it with an explicit click. The safety rules are the same; the switch is just done by a person instead of automatically.

When something goes wrong

Situation	What happens	How it is fixed
The file contains malware	Quarantined; never read or published	Upload a clean file.
The file cannot be read, or contains Arabic	Failed, with the reason and location	Fix the file and retry reading, or upload a new version.
A blocking extraction warning	Approval is refused while the affected block is included	Exclude that block, or fix the file.
Someone else changed the document meanwhile	The upload, review or approval is refused	Reload and try again on the current version.
The embedding provider fails during indexing	The attempt fails; waits 30 seconds per attempt and tries again; stops after 3 attempts	An admin retries indexing once the provider is back.
The worker crashes mid-task	Its 11-minute lease runs out	The task is claimed again automatically; finished batches are reused.

Withdrawing a document

The owner (or an admin acting for them) can withdraw a document with a reason. Its approval is marked withdrawn, its chunks stop being eligible for search immediately, an event is written, and its citations stop being current. The document and its history are kept.

4. What keeps both flows safe and correct

Safeguard	Where	What it protects against
Service token on /internal routes	Flow A, step 3	Anyone else reading the library.
Sign-in and the knowledge_admin role	Flow B, steps 1 and 2	Uploads or approvals by the wrong people.
Malware scan before reading	Flow B, step 6	Hostile files reaching the readers.
Readers in a bounded separate process	Flow B, step 7	A broken file exhausting the service.
Human review of every block	Flow B, step 9	Misread or private text being published.
Fingerprint on approval	Flow B, step 10	Approving something other than what was reviewed.
Version numbers on every change	Flow B, steps 3, 9, 10	Two people overwriting each other.
Staged chunks and one atomic switch	Flow B, steps 16 and 17	Searchers seeing a half-built or mixed version.
Verification of every search result	Flow A, step 11	Handing out text that differs from what was approved.
Stable ordering of equal scores	Flow A, step 11	The same question ranking differently on different days.
Events in the same transaction	Flow B, step 18	The requirement portal missing a change.
Current-passage check	Flow A, step 16	Relying on a passage that was replaced.
English-only rule	Flow A step 5; Flow B steps 2, 7, 9	Text the library is not set up to search.

Good to know

Every change to how chunks are cut, embedded or searched is measured on the library's golden set before it ships (docs/operations/library-retrieval-evaluation.md). Today, with the offline test models, it finds 95% of the expected passages in the top 10, and the first right passage is usually at the top (mean reciprocal rank 87.5%). These are test-model figures; the real Gemini model has not been measured yet.

5. Glossary

Term	Meaning
Block	A piece of extracted text with its location: a paragraph, table row, slide paragraph or page.
Chunk	A small piece of an approved block: the unit that is embedded, searched and cited.
Citation	A pointer from an answer to the exact approved words it relies on.
Content hash	A fingerprint of a chunk's searched text, the same every time for the same text.
Embedding	A list of numbers (here 768) that places a text on a map of meaning.
Event outbox	A table of change events, written together with each change, that another service reads.
Extractor	A program that reads a file format and pulls out its text.
Fingerprint	A short value computed from content; any change to the content changes it.
HNSW	An index that finds the nearest vectors quickly.
Idempotency key	A unique id for an upload attempt, so a retry does not create a duplicate.
Lease	A time-limited reservation of a task, so a crashed worker's task is picked up again.
Normalisation	Making text uniform (lower case, single spaces) before comparing it.
Publication	One approval of one reviewed version; the one in service is what search uses.
Reciprocal rank fusion	Merging rankings by adding 1/(60 + rank) from each list.
Revision	One saved review of a version: which blocks are included, with what text.
Service token	A secret credential one service uses to call another.
Token	A small piece of text; we count about 3.8 characters of English per token.
Vector	An ordered list of numbers; here, an embedding.